

Purpose and delivered scope

This portfolio application helps an agent combine a customer message with screenshots, PDF documents and optional voice input. It retrieves approved policy excerpts, displays source citations and saves an agent-reviewed ticket. All policies and sample customer inputs are fictional.

The baseline runs locally with Python and a browser. TF-IDF sparse vectors support retrieval; the default response contains exact evidence excerpts. Optional neural embeddings, audio transcription and Ollama generation are implemented integrations that require separately downloaded models. They were not live-tested here.

The system does not send customer messages, perform refunds or connect to a live CRM. Public deployment is pending. The project repository is `A-Multimodal-AI-Customer-Support-Copilot` under `harshakavali81-collab` on GitHub. Treat it as a local portfolio demo, not a production customer service installation.

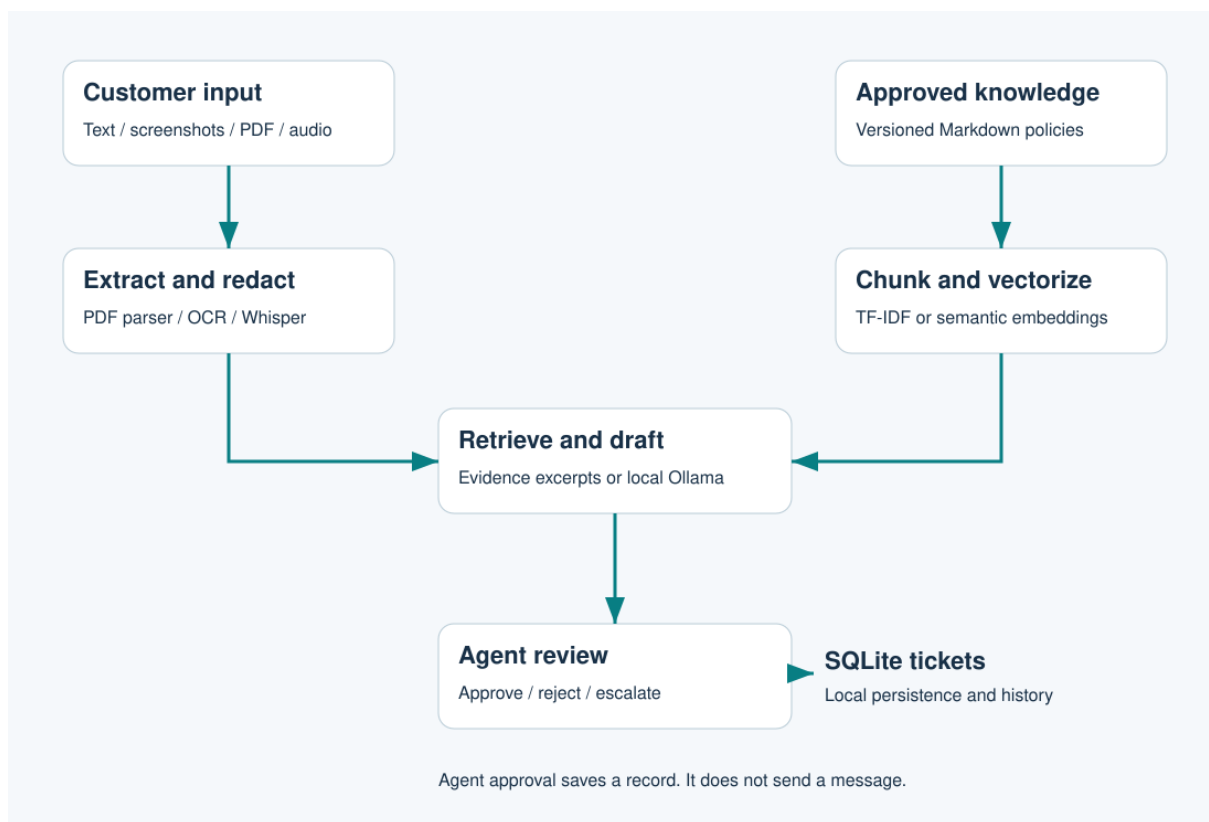
Business use cases include failed payments, password resets, refund reviews, delivery queries, subscription cancellation, file upload errors and account security. The objective is traceability and useful agent assistance. No business impact or job-selection guarantee is claimed.

Setup and first run in VS Code

1. Extract the ZIP and open the support-copilot folder in VS Code. Install Python 3.11 or 3.12. Open Terminal > New Terminal.
2. On Windows run: `py -m venv .venv`. Then run: `.venv\Scripts\python.exe -m pip install -r requirements.txt`. Calling the virtual-environment executable directly avoids PowerShell activation issues.
3. Run: `.venv\Scripts\python.exe app.py`. Open `http://127.0.0.1:8000`. Enter: My payment failed with PAY-402. Read the cited policy, edit the proposed answer and click Approve or Escalate.
4. Upload `data/samples/customer_case.pdf` to test PDF extraction. For screenshots, install Tesseract and confirm `tesseract --version` works. Then upload `data/samples/payment_error.png`. TXT and Markdown inputs also work.
5. Refresh history to see the saved ticket. Approval saves a local decision only. Stop the app using Ctrl+C. Delete `runtime/tickets.db` to reset local demo history.

On macOS/Linux use `python3 -m venv .venv`, then `.venv/bin/python -m pip install -r requirements.txt` and `.venv/bin/python app.py`. Initial dependency installation requires internet access.

Architecture and workflow



Customer inputs pass through extraction and redaction. Approved knowledge follows a separate ingestion path: Markdown policies are split into 160-word chunks with 30-word overlap, then vectorized. Uploaded customer documents are not automatically trusted as policy.

At startup the application builds an in-memory vector index. TF-IDF is the default sparse representation. The optional sentence-transformer backend creates dense embeddings. This demo does not use a persistent external vector database.

The retrieval layer selects at most three chunks above its similarity threshold. Each chunk carries a source filename and identifier, such as payments:1. Similarity scores are not probabilities or calibrated confidence.

If evidence is missing, the app recommends escalation. Otherwise it returns cited excerpts or requests an optional local LLM draft. The agent reviews the result and saves an approval, rejection or escalation in SQLite.

Multimodal and model configuration

Text files are decoded as UTF-8. PDF text extraction uses pypdf and preserves page labels. Scanned PDFs should be converted to PNG images before uploading. Screenshot processing uses Tesseract OCR, which reads visible text and error codes; it is not general visual reasoning.

Audio: run `python -m pip install -r requirements-audio.txt`. Record the supplied `voice_transcript.txt` on your phone and upload the recording. `faster-whisper` uses the tiny model on CPU with int8 computation. First use may download model weights. Supported extensions include WAV, MP3, M4A, OGG and FLAC.

Neural search: install `requirements-semantic.txt` and set `SEMANTIC_SEARCH=1` before launching. The default optional model is `all-MiniLM-L6-v2`. Model weights need internet access initially. Evaluate this mode separately because similarity thresholds differ from TF-IDF.

LLM mode: install Ollama, download a supported chat model, and set `OLLAMA_MODEL` to the installed model name. Start Ollama locally and enable Use local Ollama drafting. The adapter calls `http://127.0.0.1:11434/api/chat`. On PowerShell use `$env:OLLAMA_MODEL="your-installed-model"`; on bash use `export OLLAMA_MODEL=your-installed-model`.

If the model is unavailable or citation IDs fail validation, the app explicitly falls back to excerpts. Valid IDs alone do not prove that generated claims are supported. The agent must check the evidence. Live audio, neural embedding and Ollama inference remain unverified in this environment.

Files, interfaces and persistence

app.py serves the local application. web/index.html contains the agent workspace. copilot/ingest.py handles files. copilot/core.py owns chunking, redaction, retrieval and optional drafting. copilot/store.py owns SQLite persistence.

data/kb holds approved fictional policies. data/samples includes a screenshot, PDF, text message and audio recording script. data/evaluation.json defines ten retrieval cases. scripts/evaluate.py writes reports/evaluation.json. tests/test_core.py verifies core behavior.

GET /health checks availability. POST /api/assist accepts message, attachments and use_llm. Attachments contain name and base64 data. The response includes draft, sources, mode, status, latency_ms and ticket_id.

POST /api/review accepts id, status and reply. Allowed decisions are approved, rejected and escalated. Approval requires a non-empty reply. GET /api/tickets returns up to 100 recent records. API calls require the per-process X-Copilot-Token supplied by the local page.

The token is a local request barrier, not multi-user authentication. The SQLite tickets table stores ID, creation time, status, result JSON and edited reply. Email and number-like patterns are masked heuristically. The system does not retain uploaded file bytes after extraction.

Evaluation, limitations and monitoring

The baseline passed 10 of 10 synthetic retrieval cases and 10 unit tests in the recorded run. The retrieval check compares the highest-ranked source with the expected chunk or checks that an unsupported request produces no source. Results are in reports/evaluation.json.

The suite covers payment retrieval, unsupported queries, a narrow injection pattern, email/number redaction, empty input, chunk overlap, file type rejection, text extraction, ticket updates and non-empty approval. It does not establish real-world answer accuracy or comprehensive security.

Rerun `python -m unittest discover -s tests -v` and `python scripts/evaluate.py` after changes. The workbook contains the measured cases, formula-driven pass checks and delivery status. Its latency values measure the answer function only and exclude extraction and model startup.

Limits: three files per request, 10 MB per file, 30 PDF pages, 20-megapixel images and two-minute audio. The server is single-process and synchronous. Redaction and injection checks are heuristic. Upload errors should trigger agent follow-up, not fabricated answers.

Before production add user authentication, role-based document access, rate limits, robust file isolation, encrypted storage, retention controls and background jobs. Monitor end-to-end latency, source coverage, agent rejection rates and model failures. Use a held-out test set with paraphrases, ambiguous cases and conflicting policies. No public monitoring service is deployed.

GitHub and local deployment

Repository:

<https://github.com/harshakavali81-collab/A-Multimodal-AI-Customer-Support-Copilot> . The repository contains source, tests, diagrams, PDF documentation and the evaluation workbook.

For a new empty repository, run `git init`, `git add .`, `git commit -m "Build support copilot"`, `git branch -M main`, `git remote add origin YOUR_REPOSITORY_URL`, then `git push -u origin main`. Use normal GitHub authentication. Never put tokens in source files.

If the repository already has commits, clone it first and copy the project into the clone. Do not force-push over existing work. The `.gitignore` excludes runtime records, virtual environments and `.env` files. The GitHub Actions workflow runs unit tests and baseline evaluation.

Optional Docker commands: `docker build -t support-copilot .` and `docker run --rm -p 127.0.0.1:8000:8000 support-copilot`. The port is exposed only on host loopback. Docker was not built here. Audio and Ollama need additional model/container configuration.

Update approved Markdown policies, restart to rebuild the index, then rerun evaluation. File names and chunk order affect citation identifiers. Update expected test sources when policies change. Keep private customer documents out of the knowledge-base folder.

Interview demo and further work

Demo 1: Submit PAY-402 payment failed with the screenshot. Explain how OCR extracts the code and how the knowledge search finds payments:1. Show the actual source text before approving the reply.

Demo 2: Upload customer_case.pdf and demonstrate the same source-grounded flow. Edit the reply and approve it, then refresh history. Explain that approval is recorded locally and does not send a customer message.

Demo 3: Ask quantum banana spaceship. Show escalation when relevant guidance is absent. Explain the limits of lexical retrieval and how semantic embeddings may help paraphrases after evaluation.

Resume wording after running and understanding the project: Built a Python support copilot with PDF extraction, screenshot OCR, cited policy retrieval and SQLite agent review. Added optional local speech transcription and LLM adapters with citation validation and fallback behavior.

Explain tradeoffs honestly: the default is extractive, model integrations require setup, citations need review, the sample evaluation is small and deployment is local. Future work includes a real CRM connector, authenticated teams, multilingual evaluation and model-specific quality benchmarks.

Official implementation references: <https://docs.ollama.com/api/chat> ;
<https://github.com/SYSTRAN/faster-whisper> ; <https://www.sbert.net/> ;
https://scikit-learn.org/stable/modules/feature_extraction.html ; <https://pypdf.readthedocs.io/>
; <https://tesseract-ocr.github.io/> .